



LeetCode 258: Add Digits

1. Problem Title & Link

Title: LeetCode 258: Add Digits

Link: <https://leetcode.com/problems/add-digits/>

2. Problem Statement (Short Summary)

Given an integer num, repeatedly add all its digits until the result contains only one digit.

Return that final single-digit number.

Example

38

$$3 + 8 = 11$$

$$1 + 1 = 2$$

Answer:

2

3. Examples (Input → Output)

Example 1

Input:

num = 38

Output:

2

Explanation:

38

$$3 + 8 = 11$$

$$1 + 1 = 2$$

Example 2

Input:

num = 0

Output:

0

**Example 3**

Input:

num = 999

Output:

9

Explanation:

$9 + 9 + 9 = 27$

$2 + 7 = 9$

Example 4

Input:

num = 12345

Output:

6

Explanation:

$1+2+3+4+5 = 15$

$1+5 = 6$

4. Constraints

$0 \leq \text{num} \leq 2^{31} - 1$

Follow-Up:

Can you solve it in $O(1)$ time?

5. Core Concept (Pattern / Topic)**Mathematics / Pattern Recognition**

Secondary Topics:

- Digit Extraction
- Modulo
- Digital Root

6. Thought Process (Step-by-Step Explanation)**Brute Force Thinking**

Keep summing digits until the number becomes a single digit.

Example:

38



$$3 + 8 = 11$$

$$1 + 1 = 2$$

Answer:

2

How to Extract Digits?

For any number:

num = 538

Last digit:

$$538 \% 10 = 8$$

Remove last digit:

$$538 / 10 = 53$$

Repeat until number becomes 0.

Optimized Observation

Let's observe:

Number	Result
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	1
11	2
12	3
18	9
19	1
20	2

Notice a pattern?

Results repeat every 9 numbers.



This is called the **Digital Root**.

Digital Root Formula

If:

$\text{num} = 0$

Answer:

0

Otherwise:

$(\text{num} - 1) \% 9 + 1$

This gives the answer directly.

7. Visual / Intuition Diagram (ASCII Diagram)

Example:

38

3 + 8

↓

11

1 + 1

↓

2

Another Example:

999

9+9+9

↓

27

2+7

↓

9

Digital Root Cycle

1 → 1

2 → 2

3 → 3

4 → 4

5 → 5

6 → 6

7 → 7

8 → 8

9 → 9



```
10 → 1
11 → 2
12 → 3
...
18 → 9

19 → 1
```

8. Pseudocode (Language Independent)

Iterative Solution

```
while num >= 10

    sum = 0

    while num > 0

        sum += num % 10

        num = num / 10

    num = sum

return num
```

O(1) Mathematical Solution

```
if num == 0
    return 0

return (num - 1) % 9 + 1
```

9. Code Implementation

Python (Iterative)

```
class Solution:
    def addDigits(self, num: int) -> int:

        while num >= 10:

            digit_sum = 0
```



```
while num > 0:

    digit_sum += num % 10
    num //= 10

    num = digit_sum

return num
```

Python (Optimal)

```
class Solution:
    def addDigits(self, num: int) -> int:

        if num == 0:
            return 0

        return (num - 1) % 9 + 1
```

Java (Iterative)

```
class Solution {

    public int addDigits(int num) {

        while(num >= 10) {

            int sum = 0;

            while(num > 0) {

                sum += num % 10;
                num /= 10;
            }

            num = sum;
        }

        return num;
    }
}
```

Java (Optimal)



```
class Solution {  
  
    public int addDigits(int num) {  
  
        if(num == 0)  
            return 0;  
  
        return (num - 1) % 9 + 1;  
    }  
}
```

10. Time & Space Complexity

Iterative Solution

Time Complexity

$O(\log n)$

Reason:

Process each digit.

Space Complexity

$O(1)$

Mathematical Solution

Time Complexity

$O(1)$

Space Complexity

$O(1)$

11. Common Mistakes / Edge Cases

Mistake 1

Forgetting:

$num = 0$

Digital root formula fails unless handled separately.

Mistake 2

Using:

$num \% 9$

directly.

Example:



$9 \% 9 = 0$

Expected:

9

Need special handling through:

$(\text{num} - 1) \% 9 + 1$

Edge Cases

Input	Output
0	0
5	5
9	9
10	1
38	2
999	9

12. Detailed Dry Run (Step-by-Step Table)

Input:

num = 38

First Iteration

num	digit	sum
38	8	8
3	3	11
0	-	11

New num:

11

Second Iteration

num	digit	sum
11	1	1
1	1	2
0	-	2

New num:

2

Since:

$2 < 10$

Return:



2

13. Common Use Cases (Real-Life / Interview)

Checksum Systems

Used in:

Validation algorithms

Error detection

Numerology Applications

Repeated digit reduction.

Mathematical Pattern Recognition

Helps identify:

Modulo cycles

Repeating sequences

Interview Purpose

Tests:

Digit extraction

Pattern observation

Mathematical optimization

14. Common Traps (Important!)

Trap 1

Not noticing the Digital Root pattern.

Many candidates stop at the iterative solution.

Trap 2

Using recursion unnecessarily.

Trap 3

Writing:

$\text{num} \% 9$

instead of:

$(\text{num} - 1) \% 9 + 1$

Trap 4



Missing:

num = 0

special case.

15. Builds To (Related LeetCode Problems)

Progression:

LC 258 - Add Digits



LC 9 - Palindrome Number



LC 7 - Reverse Integer



LC 66 - Plus One



LC 415 - Add Strings

These improve number manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Recursion	$O(\log n)$	$O(\log n)$	Fair
Iterative Digit Sum	$O(\log n)$	$O(1)$	Good
Digital Root Formula	$O(1)$	$O(1)$	Best

Recursive Solution

```
def addDigits(num):  
  
    if num < 10:  
        return num  
  
    total = 0  
  
    while num:  
        total += num % 10  
        num //= 10  
  
    return addDigits(total)
```



Digital Root Solution

$(\text{num} - 1) \% 9 + 1$

Most optimal.

17. Why This Solution Works (Short Intuition)

Repeatedly summing digits eventually produces the number's **digital root**.

Mathematically, the digital root follows a repeating cycle based on modulo 9, allowing us to compute the answer directly using:

$(n-1) \% 9 + 1$

with a special case for $n = 0$.

18. Variations / Follow-Up Questions

Follow-Up 1

Find digital root in a different base.

Example:

Base 2

Base 16

Follow-Up 2

Keep multiplying digits instead of adding.

Example:

39

$$3 \times 9 = 27$$

$$2 \times 7 = 14$$

$$1 \times 4 = 4$$

Answer:

4

Follow-Up 3

Count how many iterations are required to reach a single digit.

Example:

38

Iterations:

$38 \rightarrow 11 \rightarrow 2$



Answer:

2 iterations

Follow-Up 4

Find the digital root for every number from:

1 to N

efficiently.

Interview Takeaway

This is one of the classic examples where:

Brute Force Solution



Pattern Recognition



Mathematical Formula

A candidate who discovers the Digital Root optimization demonstrates strong problem-solving skills beyond simple coding.